

FSRCA: Fast Super Resolution with Residual Channel Attention

Lucio Luque Materazzi, Teo Manuel Kaucher

lluquematerazzi@udesa.edu.ar
tkaucher@udesa.edu.ar

Ingeniería en Inteligencia Artificial
Visión Artificial

Otoño 2025

Resumen

Se presenta FSRCA (Fast Super Resolution with Residual Channel Attention), un modelo propio para la mejora de resolución de imágenes que combina la eficiencia de FSRCNN con bloques residuales complementados por canales de atención. Partiendo de la extracción y reducción de dimensiones propuesta en FSRCNN (de 56 a 12 canales), sustituimos el tradicional mapeo de cuatro convoluciones por un bloque en cascada RIR compuesto por dos o cuatro RCABs, permitiendo un modelado más fino de las características sin penalizar significativamente la velocidad de inferencia. Entrenado inicialmente sobre T91 y ajustado mediante fine-tuning en General100, optimizado bajo una función L1 mediante ADAM y aplicando early stopping. Las pruebas en Set5 y Set14 con factor de escala 2 muestran valores de PSNR 34.64 dB y 30.64 dB, y SSIM 0.9498 y 0.9000 respectivamente, superando a SRCNN en SSIM y FSRCNN en ambas, manteniendo una calidad perceptual de reconstrucción superior. Estos resultados validan la introducción de canales de atención en arquitecturas ligeras y exponen su potencial para escalas mayores y métricas perceptuales avanzadas.

1. Introducción

La Super-Resolución de Imágenes (SR) consiste en generar versiones de mayor resolución y nitidez a partir de originales de baja calidad, un problema sub-especificado, que quiere decir que hay diversas imágenes de alta resolución correspondientes a la de baja resolución.

Este desafío es relevante en el campo de visión artificial con aplicaciones que van desde la mejora de imágenes médicas o satelitales, como el preprocesamiento en diversos problemas, hasta tecnologías de super-resolución en tiempo real como DLSS de NVIDIA, empleadas para mejorar texturas en videojuegos. Las técnicas clásicas de interpolación (vecinos más cercanos, bilinear, bicúbica) ofrecen soluciones rápidas pero con resultados limitados en la preservación de bordes y texturas. Con la llegada de las redes neuronales convolucionales (CNNs), modelos como SRCNN demostraron que es posible aprender la transformación no lineal necesaria para reconstruir detalles finos, incrementando de manera notable tanto las métricas objetivas (PSNR, SSIM) como la percepción visual.

Este informe describe primero la replicación e implementación de las arquitecturas SRCNN y FSR-CNN en un ambiente reducido para factores de es-

cala 2 y 4. A continuación, se presenta una nueva propuesta: FSRCA (Fast Super Resolution with Residual Channel Attention), que combina la eficiencia de FSRCNN con bloques residuales en cascada y canales de atención, buscando un equilibrio entre precisión de reconstrucción y velocidad de inferencia.

El informe se estructura en cuatro partes: el marco teórico, que expone las métricas utilizadas y avances relevantes en Super Resolución; el desarrollo de los distintos experimentos; los resultados obtenidos para los modelos implementados, incluyendo comparaciones y análisis; y, finalmente, las conclusiones basadas en los resultados y consideraciones para futuros trabajos.

2. Marco teórico

2.1. Métricas de entrenamiento

El proceso de entrenamiento de un modelo consiste en ajustar sus pesos buscando minimizar alguna métrica seleccionada. El principal requisito de estas métricas es que sean diferenciables, es decir, que se puedan calcular sus derivadas con respecto a los parámetros del modelo para poder llevar a cabo la optimización. A continuación se presentan las dos

métricas de entrenamiento usadas en este informe.

Mean Squared Error (MSE) mide el promedio del cuadrado de las diferencias punto a punto entre un par de objetos. En el caso de las imágenes, la diferencia es píxel a píxel y se expresa de la siguiente manera:

$$MSE = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I(i, j) - K(i, j)]^2 \quad (1)$$

con I y K dos imágenes de dimensión $m \times n$.

Mean Absolute Error (MAE o L1) es similar a MSE en el sentido de que promedia diferencias punto a punto, pero L1 suma el módulo absoluto de las diferencias:

$$L1 = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} |I(i, j) - K(i, j)| \quad (2)$$

La diferencia en el uso del cuadrado o el módulo lleva a que MSE penalice de una manera más abrupta los errores más grandes, mientras que L1 los contempla todos por igual.

Como se puede observar a partir de las fórmulas, una minimización de las métricas equivale a una mayor similitud entre imágenes, llegando a un valor de 0 cuando ambas imágenes (*ground truth* y predicción) son iguales.

Estas métricas son ampliamente utilizadas en el ámbito de la superresolución de imágenes; sin embargo, no son las únicas. Existen otras funciones de pérdida relevantes, como la Perceptual Loss basada en VGG-19, la pérdida de Charbonnier y la pérdida adversarial, entre otras.

2.2. Métricas de evaluación

Una vez entrenado el modelo, las métricas de evaluación buscan reportar el rendimiento del modelo sobre datos no vistos durante el entrenamiento. En las tareas de Super Resolución (SR) se suelen utilizar dos métricas específicas.

La primera métrica es *Peak Signal-to-Noise Ratio* (PSNR)¹, que estima la relación entre la potencia máxima posible de una señal y la potencia del ruido que afecta la fidelidad de su representación. Suele expresarse en escala logarítmica en decibelios [dB] a partir del MSE, por medio de la siguiente fórmula:

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \quad (3)$$

donde MAX_I es el valor máximo posible de un píxel en la imagen, siendo por lo general igual a 255 o 1 si la imagen está normalizada.

La segunda métrica es el Índice de similitud estructural (SSIM)². A diferencia del PSNR, que se

basa únicamente en el error píxel a píxel, SSIM busca cuantificar la similitud percibida entre dos imágenes teniendo en cuenta la información estructural. Esta métrica evalúa no solo diferencias de intensidad, sino también de estructura $s(x, y)$, luminancia $l(x, y)$ y contraste $c(x, y)$.

Dados dos parches de imagen x e y , el SSIM se define como:

$$SSIM(x, y) = [l(x, y)]^\alpha \cdot [c(x, y)]^\beta \cdot [s(x, y)]^\gamma \quad (4)$$

donde comúnmente se utiliza $\alpha = \beta = \gamma = 1$, lo cual simplifica la ecuación a:

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (5)$$

siendo μ_x, μ_y las medias locales de los parches, σ_x^2, σ_y^2 sus varianzas locales, σ_{xy} la covarianza local y C_1, C_2 constantes pequeñas para estabilizar la división cuando el denominador es cercano a cero.

2.3. Datasets

En este informe, para el entrenamiento de los modelos SR se utilizan dos conjuntos de imágenes distintos: T91 contiene 91 imágenes de bajo tamaño y fue compilado para modelos tempranos, mientras que General100 consiste de 100 imágenes de mayor calidad sin compresión.

Por otro lado, la validación de los modelos SR se realiza sobre dos conjuntos estándar. Set5 se conforma de 5 imágenes cuidadosamente seleccionadas por su riqueza en bordes y texturas. En cambio, Set14 presenta imágenes de mayor diversidad y complejidad visual.

Para abordar este problema existen otros conjuntos de datos para entrenar (ImageNet) y para evaluar (BSD100, BSD200).

Aunque a primera vista los datasets de superresolución parecen contener un número reducido de imágenes, en realidad se le aplican técnicas de data augmentation. Cada fotografía se somete a rotaciones y redimensionamientos con diversos factores de escala; a partir de cada versión resultante se extraen pequeños parches de baja resolución (LR, low resolution) y de alta resolución (HR, high resolution). De este modo, a partir de una única imagen es posible generar miles de pares LR–HR, incrementando de forma considerable tanto la cantidad como la variedad de muestras disponibles para entrenar los modelos.

2.4. Interpolación

La interpolación³ es una técnica usada en tareas como el *zooming* (ampliación), *shrinking* (reducción), rotación y corrección geométrica en imágenes digitales. En este trabajo será utilizada para las dos primeras.

¹https://en.wikipedia.org/wiki/Peak_signal-to-noise_ratio

²<https://ece.uwaterloo.ca/~z70wang/publications/ssim.pdf>

³Más información en Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson. Ver Sección 2.4: *Image Sampling and Quantization*, apartado *Image Interpolation*.

En términos generales, interpolar consiste en utilizar datos conocidos para estimar valores en ubicaciones desconocidas. Al aumentar o reducir el tamaño de una imagen, se genera una nueva grilla de píxeles que puede no coincidir con los puntos originales, por lo que es necesario asignar valores de intensidad estimados para construir la nueva imagen.

Existen diversos métodos de interpolación, pero en este trabajo se utilizarán tres de los más comunes: vecinos más cercanos, bilineal y bicúbica.

Por un lado, la Interpolación de vecinos más cercanos (*nearest-neighbor interpolation*) asigna a cada nueva posición de pixel el valor de intensidad del pixel más cercano en la imagen original. Este enfoque es simple y computacionalmente eficiente, pero suele producir artefactos visuales indeseables como severas distorsiones de bordes rectos, debido a que no se generan nuevos valores intermedios, sino que se replican los existentes.

Por otro lado, la interpolación bilineal mejora considerablemente la calidad visual al considerar los cuatro vecinos más cercanos (superior izquierdo, superior derecho, inferior izquierdo e inferior derecho) del punto al que se quiere asignar un nuevo valor. El valor de intensidad $v(x, y)$ en la posición (x, y) se estima mediante una combinación bilineal de la forma:

$$v(x, y) = ax + by + cxy + d \quad (6)$$

Donde los coeficientes a , b , c y d se determinan resolviendo un sistema de cuatro ecuaciones obtenidas a partir de las intensidades de los cuatro píxeles vecinos. Este método logra mejores resultados que la interpolación de vecinos más cercanos con un leve incremento en los recursos computacionales.

Por último, la interpolación bicúbica involucra los dieciséis píxeles más cercanos (una grilla de 4×4 alrededor del punto). El valor asignado al punto (x, y) es obtenido utilizando la siguiente ecuación cúbica bidimensional:

$$v(x, y) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij} x^i y^j \quad (7)$$

Los dieciséis coeficientes a_{ij} están determinados por las intensidades de los dieciséis píxeles vecinos. Generalmente, este método realiza un mejor trabajo en preservar los detalles finos que la interpolación bilineal generando imágenes con transiciones más suaves y generales pero a un mayor costo computacional.

Existen otros métodos más sofisticados, como interpolaciones basadas en *splines* o *wavelets*, que consideran más vecinos para interpolar o utilizan modelos matemáticos más complejos. Sin embargo, por su eficiencia y balance entre costo y calidad, los métodos mencionados son los más utilizados en tareas prácticas de super-resolución.

En el trabajo además se menciona la interpolación por área, diseñada específicamente para el submuestreo de imágenes (*downsampling*). En este método se calcula el valor de cada nuevo píxel como el promedio ponderado de los píxeles originales que caen dentro del área correspondiente. Según la documentación oficial de la biblioteca OpenCV⁴, este tipo de interpolación es preferible en problemas de SR porque genera resultados sin efecto muaré, un patrón visual artificial que aparece cuando se reducen patrones repetitivos como rayas o telas.

2.5. SRCNN

El modelo de red convolucional SRCNN⁵ (Super-Resolution Convolutional Neural Network) fue propuesto por Dong et al. en 2014 y cambió el paradigma de la super-resolución al ser la primera aproximación en emplear redes neuronales convolucionales profundas (CNNs). A diferencia de los métodos tradicionales que recurrían a técnicas de interpolación o a algoritmos de reconstrucción iterativa, SRCNN plantea un modelo de extremo a extremo capaz de aprender el mapeo no lineal entre imágenes de baja y alta resolución directamente a partir de los datos, incluso llegando a mejores resultados en

El pipeline general de SRCNN consta de tres etapas principales, implementadas como capas convolucionales sucesivas:

Extracción de características: A partir de la imagen de baja resolución (previamente interpolada bicúbicamente a la dimensión deseada), se extraen representaciones de alto nivel a través de una convolución con múltiples filtros. Formalmente, esta etapa actúa como una operación de mapeo no lineal $F_1(Y) = \max(0, W_1 * Y + B_1)$ donde Y es la imagen interpolada, W_1 los filtros y B_1 los sesgos.

Mapeo no lineal: Las características extraídas se transforman mediante otra capa convolucional que refina las representaciones intermedias, ajustándolas al espacio de características de la imagen de alta resolución. Se aplica una segunda convolución seguida de una función de activación ReLU.

Reconstrucción: Finalmente, una tercera capa convolucional genera la imagen de alta resolución estimada a partir de las representaciones internas, reensamblando la información en el dominio de la imagen.

Los tamaños de los kernels de convolución en cada capa son los siguientes: $f_1 = 9$, $f_2 = 3$, $f_3 = 5$ (extracción, mapeo y reconstrucción, respectivamente). Con respecto a f_2 , en el paper original se realizó un estudio en el que se fijaron $f_1 = 9$ y $f_3 = 5$ probando con (i) $f_2 = 1$, (ii) $f_2 = 3$ y (iii) $f_2 = 9$ en factor de escala 3. Si bien encontraron que el PSNR aumentaba con un kernel más grande, la diferencia era marginal ((i) = 32,52 dB, (ii) = 32,66 dB, (iii) = 32,75 dB) y generaba un gran aumento en la

⁴https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html#gga5bb5a1fea74ea38e1a5445ca803ff121acf959dca2480cc694ca016b81b442ceb

⁵<https://arxiv.org/abs/1501.00092>

cantidad de parámetros ((i) = 8,032, (ii) = 24,416, (iii) = 57,184), por ende en el tiempo y complejidad del modelo.

La cantidad de filtros utilizados en las capas intermedias son $n_1 = 64$, $n_2 = 32$. Dong et al. también realizaron otro estudio sobre este parámetro, comparando PSNR y velocidad de restauración para un factor 3 sobre el conjunto de validación Set5: con $n_1 = 128$, $n_2 = 64$ 32,60 dB y 0.6 segundos, con $n_1 = 64$, $n_2 = 32$ 32,52 dB y 0.18 segundos y con $n_1 = 32$, $n_2 = 16$ 32,26 dB y 0.05 segundos.

Por lo tanto, concluyen que la elección de la escala de la red neuronal siempre debe ser un *trade-off* entre performance y velocidad.

Además, realizaron otros experimentos aumentando la profundidad de la red neuronal para estudiar su desempeño. Añadieron otra capa de mapeo no lineal que tiene $n = 16$ y $f = 1$ creando así las redes 9-1-1-5, 9-3-1-5, 9-5-1-5. Pero estas configuraciones no demostraron mejores valores que las pruebas anteriores, concluyendo en que no siempre ante mayor profundidad mejores resultados en estos modelos de súper resolución.

Por otro lado, en un experimento sobre canales de color se comprobó que entrenar únicamente el canal de luminancia (Y) y luego aplicar interpolación a los canales de crominancia (Cb y Cr) ofrece mejores resultados que entrenar simultáneamente los tres canales (YCbCr o RGB). Gracias a la mayor sensibilidad del ojo humano a la luminancia, esta técnica ha sido replicada en los posteriores trabajos de súper resolución.

2.6. FSRCNN

Con el objetivo de mejorar la eficiencia y superar algunas limitaciones de SRCNN, en 2016 Dong et al. propusieron FSRCNN⁶ (Fast Super-Resolution Convolutional Neural Network). A diferencia de su antecesor, este modelo evita la etapa previa de interpolación bicúbica al trabajar directamente con la imagen de baja resolución como entrada, lo que permite reducir significativamente la carga computacional.

El principal aporte de FSRCNN radica en su diseño de arquitectura: más profunda pero con menos parámetros y optimizada para lograr una aceleración significativa sin perder calidad en las métricas de reconstrucción.

La red está compuesta por las siguientes etapas:

Convolución de extracción: similar a SRCNN, extrae características iniciales a través de una convolución.

Módulo de reducción de dimensión: introduce una capa 1×1 que comprime el espacio de características, disminuyendo la cantidad de filtros. Esto reduce la complejidad de las capas siguientes.

Múltiples capas de mapeo no lineal: a diferencia de SRCNN (que tenía una sola), FSRCNN incorpora varias capas convolucionales de 3×3 para refinar las representaciones en un espacio más comprimido.

Expansión: una capa 1×1 que restaura la dimensionalidad reducida anteriormente.

Convolución transpuesta: etapa final que realiza el upsampling, generando la imagen en alta resolución. Esto permite que la red aprenda directamente la tarea de escalado, en lugar de depender de interpolaciones externas.

A diferencia de SRCNN, en vez de utilizar como función de activación a la función ReLU, esta arquitectura emplea la función PReLU para evitar las "dead features" causadas por los gradientes iguales a 0. La configuración típica evaluada en el paper (denominada 56-12-4) corresponde a: 56 filtros en la capa de extracción inicial, reducción a 12 filtros en las capas intermedias y 4 capas de mapeo no lineal.

Esta arquitectura logró resultados superiores a los de SRCNN tanto en PSNR como en SSIM, incluso con una mejora de hasta 40 veces en velocidad de inferencia sobre CPU.

Además, FSRCNN aprende el proceso de aumento de resolución como parte del entrenamiento, lo que lo hace más flexible a diferentes factores de escalado sin requerir rediseño de la arquitectura. Es decir, permite entrenar toda la red para un factor de escala y, al momento de cambiar de factor, solo reentrenar la última capa del modelo. En el paper se demuestra que este procedimiento permite una convergencia más rápida que entrenar desde cero.

2.7. ESPCN

ESPCN⁷ (Efficient Sub-Pixel Convolutional Neural Network) fue propuesto en 2016 por Shi et al., buscando un modelo eficiente que pueda operar en *real-time*. De manera similar a FSRCNN, ESPCN evita el uso de upsampling al inicio para aumentar el tamaño de las imágenes, pero la técnica planteada es distinta: al final de la red se coloca una capa denominada Sub-Pixel Convolution, que escala la imagen mediante una operación denominada PixelShuffle. Esta operación espera una entrada de dimensión $(N, C \times r^2, H, W)$ y reordena los píxeles de los canales C hacia la altura H y ancho W de la imagen, produciendo una salida de dimensión $(N, C, H \times r, W \times r)$. Esta manera de transformar la imagen, además de aumentar la velocidad de entrenamiento e inferencia debido a que se trabaja en menores dimensiones hasta el final, permite evitar la posible generación de *artefactos* (errores visuales de reconstrucción) que podría ocurrir al usar una convolución transpuesta en su lugar.

⁶<https://arxiv.org/abs/1608.00367>

⁷<https://arxiv.org/abs/1609.05158>

⁸<https://arxiv.org/abs/1807.02758>

2.8. RCAN

En 2018, Zhang et al. propusieron RCAN⁸ (Residual Channel Attention Network), una arquitectura profunda que introdujo dos avances significativos en el campo de la súper resolución: el uso extensivo de bloques residuales en cascada (Residual in Residual, RIR) y la incorporación de mecanismos de atención de canal (Channel Attention, CA) dentro de bloques residuales convolucionales.

RCAN demostró ser altamente eficaz, obteniendo grandes resultados en términos de PSNR y SSIM. Su diseño está específicamente pensado para redes muy profundas, permitiendo un entrenamiento estable y una mejora sustancial en la calidad perceptiva de las imágenes reconstruidas.

Residual in Residual (RIR) El modelo organiza su arquitectura en una jerarquía residual. En lugar de aplicar un único atajo (residual global), RCAN introduce una estructura RIR, que consiste en múltiples bloques residuales agrupados y envueltos también en una conexión residual externa. Esta composición facilita el flujo de gradientes y permite entrenar redes de cientos de capas sin degradación del rendimiento.

Formalmente, si un grupo de bloques realiza una transformación $F_G(\cdot)$, el esquema RIR toma una entrada x y produce $RIR(X) = x + F_G(x)$, donde F_G incluye, a su vez, múltiples bloques residuales internos.

Residual Channel Attention Block (RCAB) Cada bloque interno de la red RIR es un RCAB, es decir, un bloque residual que no solo aplica convoluciones y activaciones no lineales, sino que también incluye un módulo de atención para recalibrar la importancia de cada canal.

La estructura general del RCAB es: Convolución 3×3 + ReLU, Convolución 3×3 , Módulo de atención de canal (CA) y Suma residual con la entrada del bloque. Que se puede resumir en:

$$RCAB(x) = x + CA(Conv_2(ReLU(Conv_1(x)))) \quad (8)$$

Channel Attention (CA) El módulo CA es la principal innovación de RCAN. Su objetivo es otorgar mayor peso a los canales más relevantes de las representaciones intermedias. Para ello, aplica un mecanismo de atención global a lo largo del eje de canales.

Los pasos básicos del módulo CA son:

Global Average Pooling: se calcula una estadística por canal, resumiendo espacialmente la activación.

Bottleneck: se aplica una red de dos capas totalmente conectadas (con una reducción de dimensionalidad y luego expansión), activación ReLU intermedia.

Sigmoid: para obtener los pesos de atención por canal.

Reescalado: los mapas de características se multiplican canal a canal con estos pesos.

Este mecanismo permite que la red se enfoque dinámicamente en los canales más útiles para la reconstrucción de detalles finos.

2.9. Otros modelos y avances recientes

Además de los modelos mencionados anteriormente (SRCNN en 2014, FSRCNN en 2016, ESPCN en 2016 y RCAN en 2018), existieron otros modelos desarrollados para esta problemática. Entre las propuestas más influyentes se encuentran los modelos basados en aprendizaje adversarial, como SRGAN⁹ y ESRGAN (Enhanced Super-Resolution GAN, 2017-2018), que introducen una función de pérdida adversarial combinada con pérdidas perceptuales para generar imágenes visualmente más realistas, aunque a veces con un leve sacrificio en métricas tradicionales como PSNR, pero con incremento en el testeo MOS (*Mean Opinion Score*).

Por otro lado, modelos como EDSR¹⁰ (Enhanced Deep Super-Resolution Network, 2017) y MDSR (Multi-Scale Deep Super-Resolution Network, 2017), propuestos por Lim et al., eliminaron normalizaciones innecesarias y optimizaron estructuras residuales para obtener un rendimiento superior, logrando grandes resultados sin necesidad de GANs.

Finalmente, el desarrollo reciente ha llevado al surgimiento de modelos aún más avanzados como SwinIR¹¹ (Image Restoration using Swin Transformers, 2021), Real-ESRGAN¹² (2021) y HAT¹³ (Hybrid Attention Transformer, 2023) que integran transformers o estrategias de entrenamiento robustas orientadas a degradaciones reales. Estos modelos representan el estado del arte actual, destacándose tanto en calidad visual como en capacidad de generalización frente a imágenes del mundo real.

3. Desarrollo

El desarrollo general del modelo fue el siguiente: primero se buscó replicar las estructuras de los modelos SRCNN y FSRCNN para factores de escala 2 y 4, comparando con métodos de interpolación clásicos. Luego se propusieron distintos modelos propios para SR. Todos estos modelos se entrenaron para poder escalar el canal de luminancia Y de la representación $YCbCr$ de la imagen, realizando una interpolación bicúbica a los canales de color para obtener la reconstrucción completa.

⁹<https://arxiv.org/abs/1609.04802>

¹⁰<https://arxiv.org/abs/1707.02921>

¹¹<https://arxiv.org/abs/2108.10257>

¹²<https://arxiv.org/abs/2107.10833>

¹³<https://arxiv.org/abs/2205.04437>

Para el entrenamiento se definieron los siguientes hiperparámetros globales: optimizador ADAM ($\beta_1 = 0,9, \beta_2 = 0,999$), 1000 épocas, *early stopping* con paciencia de 10 épocas, métrica MSE, *learning rate* (lr) de 10^{-4} y un *lr scheduler* de tipo ReduceLROnPlateau con paciencia de 5 épocas hacia $\frac{lr}{2}$.

En cuanto a la preparación del dataset, se definió que se generarían 550 pares de parches (LR, HR) por cada imagen. La generación de cada parche consistió en un proceso de pasos aleatorios, donde la imagen podía ser previamente achicada, rotada, espejada, y cortada en una sección aleatoria. Se realizó este procedimiento para los datasets T91 y General100, separando en cada caso un 20 % de los parches para la validación del modelo.

En comparación con los modelos de SRCNN y FSRCNN implementados en los papers, el entrenamiento se realiza sobre un menor conjunto de datos con menores épocas. Estas decisiones fueron tomadas teniendo en consideración las limitaciones de extensión de tiempo para realizar el trabajo, aceptando que los modelos implementados podrían no alcanzar su rendimiento presentado en los papers originales. De esta manera, los primeros dos modelos sirven de comparación para evaluar el comportamiento del modelo propio propuesto, que también se entrena con parámetros reducidos.

En SRCNN, la arquitectura implementada fue la versión 9-3-5 con 64 y 32 las cantidades de filtros de las capas intermedias, resultando en 24.513 parámetros entrenables para el modelo. Esos parámetros fueron entrenados para factor 2 solamente sobre T91, siguiendo uno de los procedimientos del paper original.

Para FSRCNN se utilizó la arquitectura 56-12-4 que derivó en 12,809 parámetros. Se replicó su estrategia de entrenamiento, consistiendo en un primer entrenamiento sobre T91 seguido de un refinamiento (*fine-tuning*) sobre General100, para factor 2. Aprovechando que FSRCNN se puede adaptar a distintas escalas con pocos cambios, se congelaron los pesos de todas las capas menos la última, la cual se transformó para luego entrenar un FSRCNN para factor 4 aprovechando el entrenamiento de factor 2. Debido a esto solo se tuvieron que entrenar 4.537 parámetros.

Para el modelo propio se buscó priorizar el tiempo de inferencia de FSRCNN a medida que se implementaban técnicas más recientes como los mecanismos de atención de RCAN. Presentamos entonces FSRCA (Fast Super Resolution with Residual Channel Attention), nombrado a partir de la estructura que se presenta a continuación. FSRCA conserva la estructura general de FSRCNN, comenzando con una etapa de extracción de características que utiliza una capa convolucional con 56 filtros y un tamaño de kernel de 5. A continuación, se aplica una etapa de reducción dimensional (*shrink*) que achica la cantidad de canales a 12 mediante una convolución con kernel de tamaño 1. En lugar del bloque tradicional de mapeo no lineal compuesto por

cuatro capas convolucionales, FSRCA introduce un bloque de tipo RIR con dos o cuatro bloques internos RCAB, lo que permite integrar mecanismos de atención por canal sin aumentar considerablemente la complejidad computacional.

A continuación, tras volver a expandir la dimensionalidad a 56 canales con otra convolución 1×1 , incorporamos dos atajos residuales: el primero suma la entrada y la salida del bloque RIR para mejorar el flujo de gradiente en el mapeo; el segundo conecta los resultados de la extracción (primera etapa) con los resultados de la expansión para la etapa de reconstrucción (última etapa).

Para dicha fase, se implementaron y compararon dos variantes: en una se propusieron 4 bloques RCAB y se utilizó una convolución transpuesta con kernel de tamaño 9, stride correspondiente al factor de aumento, y padding adecuado para mantener la dimensión (denominado FSRCA); en la otra se seleccionaron 2 bloques RCAB y se aplicó una convolución con salida de canales igual al cuadrado del factor de aumento, seguida de una operación de PixelShuffle para reordenar espacialmente las características y una convolución final 1×1 con kernel de tamaño 3 (denominado FSRCA_ps). Esta comparación permitió evaluar el rendimiento de ambas técnicas de upsampling en términos de calidad visual y eficiencia computacional. El diseño del modelo busca alcanzar un balance entre velocidad de inferencia y fidelidad de reconstrucción, apuntando a contextos donde el tiempo de procesamiento sea una variable crítica sin resignar la riqueza del detalle visual. El modelo FSRCA_ps presenta 27,764 y 82,208 parámetros para los factores 2 y 4, mientras que FSRCA resulta en 19,449 para ambos factores.

Aunque el diseño original de RCAN emplea el esquema RIR en redes muy profundas, en esta implementación se adopta una versión liviana del mismo principio, utilizando un único grupo RIR con solo cuatro bloques RCAB. Esta decisión busca obtener parte de los beneficios de estabilidad y refinamiento que ofrece la jerarquía residual, sin comprometer la velocidad de inferencia.

A pesar de la directa relación con PSNR, para las dos implementaciones de FSRCA se decidió evitar el uso de MSE como función de pérdida porque tiende a generar imágenes suavizadas con falta de texturas perceptuales. Este efecto fue estudiado en trabajos mencionados como SRGAN y ESRGAN, donde se propone reemplazar o complementar MSE con pérdidas perceptuales para lograr resultados más realistas visualmente. Es por esto que FSRCA se entrenó con L1 como función de pérdida, buscando mejorar la calidad visual antes que el PSNR.

Habiendo entrenado las dos variantes de FSRCA para factor 2, se aplicó la técnica de FSRCNN para entrenar solo la última capa para factor 4. Los resultados se compararon tanto para los valores de PSNR y SSIM como para la calidad visual de reconstrucción.

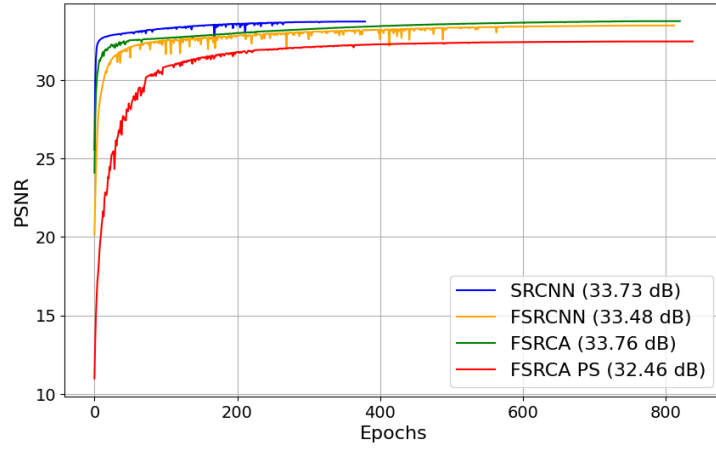


Figura 1: Comparación de PSNR de validación durante el entrenamiento para todos los modelos en factor 2

4. Resultados

Dataset	Factor	NN	Bilineal	Bicúbico
Set5	2	29.53/0.9018	30.93/0.9108	32.63/0.9323
Set14	2	27.26/0.8494	27.93/0.8410	29.22/0.8761
Set5	4	24.94/0.7448	26.24/0.7808	27.31/0.8097
Set14	4	23.42/0.6585	24.19/0.6725	24.89/0.7047

Cuadro 1: Comparación de resultados PSNR/SSIM para distintas técnicas de interpolación sobre Set4 y Set14 en factor de escala 2 y 4.

Primero se realizó una comparación para evaluar el rendimiento de los distintos métodos clásicos de interpolación. Como se puede observar en el cuadro 1, el método bicúbico consigue siempre mejores resultados por un margen significativo. Para los siguientes estudios, solo se comparan los modelos propios contra la interpolación bicúbica para simplificar las figuras y las evaluaciones.

En la figura 1 se observa cómo el entrenamiento de SRCNN difiere de los demás. Esto se debe a que SRCNN comienza trabajando con las imágenes ya interpoladas, por lo cual no necesita ajustar sus pesos para conseguir un valor de PSNR por encima de 30 dB. También es por esta razón que su entrenamiento se muestra estable y converge en la mitad de tiempo que los otros modelos.

Por otro lado, también se observa que FSRCA PS entrena a una menor velocidad que los otros modelos y no alcanza su rendimiento. Este resultado podría ser explicado con que la manera de entrenarlo no permite que el modelo aprenda a trabajar bien con PixelShuffle, que puede ser una operación menos directa que una convolución transpuesta.

En cuanto a los valores finales de validación alcanzados durante el entrenamiento, FSRCA consigue el mejor resultado, con una diferencia de 0,06 dB por sobre SRCNN y 0,28 dB por sobre FSRCNN. Sin embargo, dado que la cantidad de épocas de entrenamiento de FSRCA es aproximadamente el doble que el de SRCNN, la mejora en la calidad de reconstrucción podría no compensar la complejidad adicional en entornos con recursos de cómputo o tiempo limitados.

A partir de la figura 2 se nota que el fine-tuning de FSRCNN y FSRCA sobre General100 logra convergencia en menos épocas que el entrenamiento inicial sobre T91. Dong et al. realizan un estudio donde exponen que esta manera de entrenar es más rápida que un entrenamiento inicial sobre la unión de ambos conjuntos de datos.

Si bien el PSNR no puede ser comparado con la figura 1 ya que son dos datasets distintos, es relevante notar que los modelos puedan seguir ajustándose a datasets no vistos durante el entrenamiento. En particular, FSRCA logra aumentar en aproximadamente 1 dB entre la primera época y la última. Por otro lado, FSRCNN presenta una mayor oscilación con un menor aumento (0,1 dB) en una mayor cantidad de épocas. Esta diferencia podría llegar a indicar que el modelo propuesto tiene mayor capacidad de adaptación.

A continuación se comparan los modelos sobre los conjuntos de testeo antes y después del fine-tuning para estudiar si se produce una mejora sobre imágenes generales.

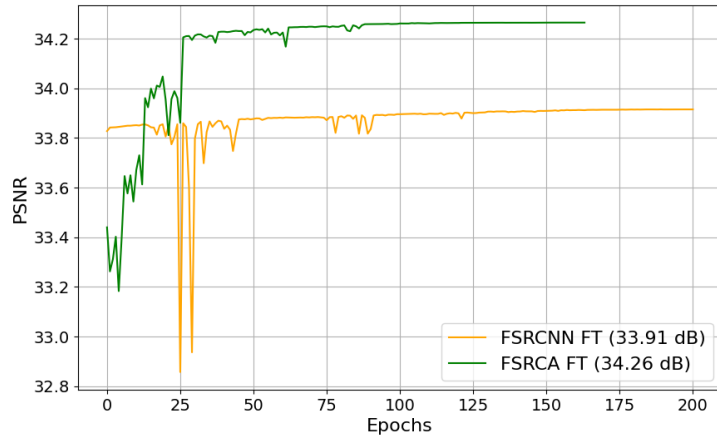


Figura 2: Comparación de PSNR de validación durante el fine-tuning para FSRCNN y FSRCA en factor 2

Dataset	Factor	FSRCNN	FSRCNN ft	FSRCA_ps	FSRCA_ps ft	FSRCA	FSRCA ft
Set5	2	34.51/0.9486	34.54/0.9488	33.40/0.9395	33.50/0.9413	34.61/0.9496	34.64/0.9498
Set14	2	30.60/0.8985	30.62/0.8988	29.85/0.8909	29.94/0.8930	30.64/0.8999	30.64/0.9000
Set5	4	28.22/0.8302	28.63/0.8467	-	-	28.45/0.8414	28.81/0.8545
Set14	4	25.58/0.7299	25.87/0.7403	-	-	25.71/0.7368	26.02/0.7464

Cuadro 2: Comparación de resultados PSNR/SSIM para los modelos implementados antes y después de realizar fine-tuning sobre Set5 y Set14 en factor de escala 2 y 4.

Observando los resultados del cuadro 2, se verifica que el fine-tuning mejora los resultados en todos los modelos estudiados. Además, se resalta que el aumento en factor 4 es consistentemente mayor al de factor 2, posiblemente debido a que esos modelos fueron entrenados por más etapas: heredan los pesos fine-tuneados de factor 2, entrenan su última capa y aplican fine-tuning una vez más.

Por un lado, se resalta que FSRCA presenta los mejores resultados antes y después para ambos factores en los dos datasets. Por otro lado, FSRCA_ps no consigue superar las métricas alcanzadas por FSRCNN para factor 2. Dado que entrenarlo para un factor 4 habría implicado más de 70,000 parámetros adicionales sin ofrecer ganancias en precisión, se optó por no implementarlo en ese escenario.

Una vez conseguido el mejor modelo de fine-tuning (FSRCA), se lo compara con la mejor interpolación (bicúbica) y el modelo sin fine-tuning (SRCNN).

Dataset	Factor	Bicúbico	SRCNN	FSRCA ft
Set5	2	32.63/0.9323	34.72/0.9496	34.64/ 0.9498
Set14	2	29.22/0.8761	30.70/0.8987	30.64/ 0.9000
Set5	4	27.31/0.8097	-	28.81/0.8545
Set14	4	24.89/0.7047	-	26.02/0.7464

Cuadro 3: Comparación de resultados PSNR/SSIM para los modelos implementados sobre Set5 y Set14 en factor de escala 2 y 4.

En el cuadro 3 se puede identificar que el método de interpolación bicúbica presenta los peores resultados, incluso al compararlo con todos los modelos de la tabla 2. Esto refuerza la idea de que los modelos que implementan CNNs obtienen mejores resultados que los métodos de interpolación clásica.

Sin embargo, se resalta que FSRCA logra el mejor valor de SSIM mientras que SRCNN el de PSNR. Este resultado se puede explicar por el hecho de que SRCNN fue entrenado para minimizar MSE, una métrica inversamente relacionada al PSNR, mientras que FSRCA utilizó L1, una métrica que preserva bordes mejor que MSE, asociándose más con la percepción estructural y SSIM.

En la figura 3 se presentan los resultados visuales de los modelos para el factor 2 sobre una imagen del set 5. En cuanto al PSNR se puede observar lo mencionado anteriormente: La interpolación bicúbica presenta el menor valor, FSRCA_ps el menor valor dentro de las CNNs y SRCNN el mejor valor entre todos los modelos. Sin embargo, para esta imagen FSRCNN presenta mejores resultados en PSNR que FSRCA, algo no observado al promediar por todas las imágenes de los datasets (tabla 2).



Figura 3: Comparación de la segunda imagen del set 5 para la interpolación bicúbica, SRCNN, FSRCNN, FSRCA_ps, FSRCA y la original para el factor 2



Figura 4: Comparación de la segunda imagen del set 5 para la interpolación bicúbica, FSRCNN, FSRCA y la original para el factor 4

En cuanto a lo perceptual, es posible visualizar que la interpolación presenta los peores resultados en comparación a las imágenes de los modelos que implementan CNNs. Esto es evidente en el parche, el cual presenta una suavización en los bordes típica de las interpolaciones. Luego, el segundo peor resultado proviene de FSRCA_ps, que tampoco logra definir correctamente los bordes. Por último, entre SRCNN, FSRCNN y FSRCA no es posible identificar una diferencia significativa para el factor 2 en la imagen 5. En la siguiente figura se evalúa si este sigue siendo el caso en factor 4.

Finalmente, a partir de la figura 4, donde se pueden ver las ampliaciones para factor 4 por parte de interpolación bicúbica, FSRCNN, y FSRCA, ya se advierte una diferencia en la calidad de reconstrucción para los métodos basados en CNNs. Esta diferencia se presenta de la forma más clara en las siguientes dos regiones: en la zona central del parche, donde se encuentra una línea oscura, se puede notar que FSRCA se acerca más a la imagen real, ya que genera menos píxeles claros en esa región que FSRCNN; en las dos secciones blancas a su derecha también se nota una reconstrucción más "sucia" por parte de FSRCNN en comparación a FSRCA. Estas mejoras en calidad perceptual parecen reflejar las diferencias en entrenamiento, así como posibles limitaciones de FSRCNN debido a su menor cantidad de parámetros o estructura limitada.

5. Conclusión

A lo largo de este informe se estudiaron distintos modelos antiguos de Super Resolución, así como técnicas provenientes de modelos más recientes. Luego de implementar SRCNN y FSRCNN, una versión que optimiza la velocidad sin perder capacidad de inferencia, se propuso un modelo propio llamado FSRCA, Fast Super Resolution with Residual Channel Attention.

El modelo combinó ideas de optimización de FSRCNN, conexiones residuales y canales de atención de RCAN y se armó una versión con la operación PixelShuffle de ESPCN, la cual rindió peores resultados.

Desde el paper de FSRCNN se implementó la metodología de entrenamiento sobre más de un set de datos a través de una etapa inicial en T91 seguida de un fine-tuning en General100. Los estudios revelaron que esta manera de ajustar los pesos llevan a una convergencia más rápida y mejoras de desempeño sobre datos no vistos.

Se demostró que las redes convolucionales superan ampliamente a los métodos de interpolación clásicos, llegando FSRCA a valores de PSNR y SSIM superiores (34.64 dB / 0.9498 en Set5 $\times 2$ y 30.64 dB / 0.9000 en Set14 $\times 2$) tras fine-tuning, sobrepasando a FSRCNN y SRCNN (solo en SSIM), incluso considerando el entrenamiento reducido. Mirando el SSIM y la calidad de las imágenes reconstruidas, también se concluyó que el entrenamiento sobre L1 en vez de MSE y la complejidad de la arquitectura de FSRCA derivaron en resultados de mayor calidad perceptual, sobre todo en factor 4.

Resumiendo los resultados obtenidos, se considera que FSRCA logra un equilibrio óptimo entre calidad de reconstrucción y eficiencia computacional.

La principal limitación del trabajo fue el tiempo de entrenamiento de los modelos. Por lo tanto, para trabajos futuros, se podrían complejizar diversas áreas del trabajo. En cuanto al proceso de entrenamiento, se sugiere entrenar sobre conjuntos de datos más diversos, con mayor cantidad de parches y épocas y probar otras métricas con un mayor enfoque en la calidad perceptual (como VGG19). Para los modelos,

se deberían realizar estudios sobre el entrenamiento del factor 4 desde cero, analizando tiempo de convergencia y rendimiento y se podrían entrenar los modelos de SRCNN y FSRCA-ps para factor 4 con el objetivo de tener mayor espacio de comparación.

6. Bibliografía

1. Wikipedia contributors. (2025). Peak signal-to-noise ratio. En Wikipedia. Disponible en: https://en.wikipedia.org/wiki/Peak_signal-to-noise_ratio
2. Wang, Z., Bovik, A. C., Sheikh, H. R. Simoncelli, E. P. (2004). Image Quality Assessment: From Error Visibility to Structural Similarity. *IEEE Transactions on Image Processing*, 13(4), 600–612. Disponible en: <https://ece.uwaterloo.ca/~z70wang/publications/ssim.pdf>
3. González, R. C. Woods, R. E. (2018). *Digital Image Processing* (4.ª ed.). Pearson. Sección 2.4: Image Sampling and Quantization, apartado Image Interpolation.
4. OpenCV Developers. (2025). Image interpolation flags – INTER_AREA. OpenCV Documentation. Disponible en: https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html#gga5bb5a1fea74ea38e1a5445ca803ff121acf959dca2480cc694ca016b81b442ceb
5. Dong, C., Loy, C. C., He, K. Tang, X. (2015). Image Super-Resolution Using Deep Convolutional Networks (SRCNN). *arXiv preprint arXiv:1501.00092*. Disponible en: <https://arxiv.org/abs/1501.00092>
6. Dong, C., Loy, C. C., He, K. Tang, X. (2016). Accelerating the Faster Super-Resolution Convolutional Neural Network (FSRCNN). *arXiv preprint arXiv:1608.00367*. Disponible en: <https://arxiv.org/abs/1608.00367>
7. Shi, W., Caballero, J., Huszár, F., Totz, J., Aitken, A. P., Bishop, R., Rueckert, D. Wang, Z. (2016). Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network (ESPCN). *arXiv preprint arXiv:1609.05158*. Disponible en: <https://arxiv.org/abs/1609.05158>
8. Zhang, Y., Li, K., Li, K., Wang, L., Zhong, B. Fu, Y. (2018). Image Super-Resolution Using Very Deep Residual Channel Attention Networks (RCAN). *arXiv preprint arXiv:1807.02758*. Disponible en: <https://arxiv.org/abs/1807.02758>
9. Ledig, C., Theis, L., Huszár, F., Caballero, J., Cunningham, A., Acosta, A., Aitken, A. P., Tejani, A., Totz, J., Wang, Z. Shi, W. (2017). Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network (SRGAN). *arXiv preprint arXiv:1609.04802*. Disponible en: <https://arxiv.org/abs/1609.04802>
10. Lim, B., Son, S., Kim, H., Nah, S. Lee, K. M. (2017). Enhanced Deep Residual Networks for Single Image Super-Resolution (EDSR MDSR). *arXiv preprint arXiv:1707.02921*. Disponible en: <https://arxiv.org/abs/1707.02921>
11. Liang, J., Lin, G., Hu, X. Yang, Q. (2021). Image Restoration Using Swin Transformer (SwinIR). *arXiv preprint arXiv:2108.10257*. Disponible en: <https://arxiv.org/abs/2108.10257>
12. Wang, X., Yu, K., Dong, C. Loy, C. C. (2021). Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data (Real-ESRGAN). *arXiv preprint arXiv:2107.10833*. Disponible en: <https://arxiv.org/abs/2107.10833>
13. Chen, X., Wang, X., Zhou, J. Dong, C. (2022). Activating More Pixels in Image Super-Resolution Transformer (HAT). *arXiv preprint arXiv:2205.04437*. Disponible en: <https://arxiv.org/abs/2205.04437>